

CS 537: Introduction to Operating Systems
Fall 2015: Midterm Exam #3
Thursday, November 19th 7:15-9:15

Persistence

This exam is closed book, closed notes.

All cell phones must be turned off.

No calculators may be used.

You have two hours to complete this exam.

Write all of your answers on the accu-scan form with a #2 pencil.

These exam questions must be returned at the end of the exam, but we will not grade anything in this booklet.

Please fill in the accu-scan form with your Last Name, First Name and Student Identification Number; remember to fill in the corresponding bubbles as well.

Good luck!

This exam has multiple versions. To make sure you are graded with the correct answer key, you must identify this exam version with a Special Code in Column A on your accu-tron sheet. Your special code is: 2. Be sure to fill in the corresponding bubble as well.

The other version of this exam simply places the True/False questions first.

Disks. [2 points each]

Consider a disk with the following characteristics:

- Number of surfaces: 8 ($= 2^3$)
- Number of tracks / surface: 512 K ($= 2^{19}$)
- Number of bytes / track: 8 MB ($= 2^{23}$ bytes)
- Number of sectors / track: 8 K ($= 2^{13}$)
- On-disk cache: 16 MB ($= 2^{24}$ bytes)

- 1) How many heads does this disk have?
 - a) 1

- b) 2
 - c) 4
 - d) 8
 - e) Not enough information or none of the above
- 2) What is the size of each sector?
- a) 512 bytes
 - b) 1KB
 - c) 2KB
 - d) 4KB
 - e) Not enough information or none of the above
- 3) How many bytes per cylinder?
- a) 1 MB
 - b) 8 MB
 - c) 64 MB
 - d) 2^{42} bytes
 - e) Not enough information or none of the above
- 4) What is the total capacity of this disk?
- a) 2^{42} bytes
 - b) 2^{45} bytes**
 - c) 2^{35} bytes
 - d) 2^{38} bytes
 - e) Not enough information or none of the above

RAID Mapping [3 points each]

The next questions ask you to translate logical read and write operations performed on top of a RAID system to the physical read and write operations that will be required to the underlying disks. Specifically, for each RAID configuration, translate the logical requests to the physical operations performed on the correct disk number and physical block address (offset). In all cases assume a block and chunk size of 4 KB. These questions are all similar to those available from the homework simulations.

9) **RAID Level: 0; Number of Disks: 8;** Random Read from logical block number 58

- a) Read from disk 2, offset 2
- c) Read from disk 7, offset 2
- d) Read from disk 7, offset 7
- e) None of the above

10) **RAID Level: 1; Disks: 8;** Random Write to logical block 29

- a) Write to disk 1 and disk 5 at offset 7
- b) Write to disk 2 and disk 3 at offset 7
- c) Write to disk 5 at offset 3
- d) Write to disk 3 at offset 5

e) None of the above

11)RAID Level: 4; Disks: 4; Random Write to logical block 50

- a) Write to disk 2, offset 12
- b) Write to disk 2, offset 16
- c) Read from disk 2, offset 16; Write to disk 2, offset 16.
- d) Read from disk 2 and 3, offset 16; Write to disk 2 and 3, offset 16.
- e) None of the above

12)RAID Level: 5 (Left Symmetric); Disks: 4; Random Read from logical block 10

- a) Read from disk 0, offset 3
- b) Read from disk 2, offset 2
- c) Read from disk 2, offset 3
- d) Read from disk 3, offset 2
- e) None of the above

The pattern for left-symmetric is:

13)RAID Level: 5 (LS); Disks: 4; Sequential Write to logical blocks 15, 16, and 17.

- a) Write to disks 0, 1, and 3 at offset 5
- b) Write to disks 0, 1, 2, and 3 at offset 5
- c) Read and write to disks 0, 1, and 3 at offset 5
- d) Read and write to disks 0, 1, 2, and 3 at offset 5
- e) None of the above

Basic File System Operations and Data Structures [3 points each]

These questions ask you to understand how different file system operations lead to different file system data structures being modified on disk. You do not need to consider journaling or crash consistency in these questions. This part is based on the available homework simulations.

This file system supports 7 operations:

- mkdir() - creates a new directory
- creat() - creates a new (empty) file
- open(), write(), close() - appends a block to a file
- link() - creates a hard link to a file
- unlink() - unlinks a file (removing it if linkcnt==0)

The state of the file system is indicated by the contents of four different data structures: inode

bitmap - indicates which inodes are allocated (not shown, because not needed for questions) inodes -

table of inodes and their contents

data bitmap - indicates which data blocks are allocated (not shown)

data - indicates contents of data blocks

The inodes each have three fields: the first field indicates the type of file (f for a regular file, d for a directory); the second indicates which data block belongs to a file (here, files can only be empty, which have the address of the data block set to -1, or one block in size, which would have a non-negative address); the third shows the reference count for the file or directory. For example, the following inode is a regular file, which is empty (address field set to -1), and has just one link in the file system: [f a:-1 r:1]. If the same file

had a block allocated to it (say block 10), it would be shown as follows: [f a:10 r:1]. If someone then created a hard link to this inode, it would then become [f a:10 r:2].

Data blocks can either retain user data or directory data. If filled with directory data, each entry within the block is of the form (name, inumber), where "name" is the name of the file or directory, and "inumber" is the inode number of the file. Thus, an empty root directory looks like this, assuming the root inode is 0: [(.,0) (.,0)]. If we add a single file "f" to the root directory, which has been allocated inode number 1, the root directory contents would then become: [(.,0) (.,0) (f,1)]

If a data block contains user data, it is shown as just a single character within the block, e.g., "h". If it is empty and unallocated, just a pair of empty brackets ([]) are shown.

Empty inodes and empty data blocks may not all be shown.

An entire file system is thus depicted as follows:

```
inode bitmap 11110000
inodes [d a:0 r:6] [f a:1 r:1] [f a:-1 r:1] [d a:2 r:2] [] ... data
bitmap 11100000
data [(.,0) (.,0) (y,1) (z,2) (f,3)] [u] [(.,3) (.,0)] [] ...
```

This file system has eight inodes and eight data blocks. The root directory contains three entries (other than "." and ".."), to "y", "z", and "f". By looking up inode 1, we can see that "y" is a regular file (type f), with a single data block allocated to it (address 1). In that data block 1 are the contents of the file "y": namely, "u". We can also see that "z" is an empty regular file (address field set to -1), and that "f" (inode number 3) is a directory, also empty. You can also see from the bitmaps that the first four inode bitmap entries are marked as allocated, as well as the first three data bitmap entries.

Assume the initial state of the file system is as follows:

```
inodes [d a:0 r:2] [] [] [] [] []
data [(.,0) (.,0)] [] [] []
```

If the file system transitions into each of the following states, what operation or operations must have been performed? The state of the file system is cumulative across questions.

14) File System State:

```
inodes [d a:0 r:3] [d a:1 r:2] [] [] [] []
data [(.,0) (.,0) (c,1)] [(.,1) (.,0)] [] []
```

- a) mkdir("/c");
- b) creat("/c");
- c) link("/", "/c");
- d) mkdir("/c/d");
- e) None of the above

15) File System State:

```
inodes [d a:0 r:3] [d a:1 r:3] [f a:-1 r:1] [] [] []
data [(.,0) (.,0) (c,1)] [(.,1) (.,0) (q,2)] [] [] a)
mkdir("/c/q");
```

- b) create("/c/q");
- c) mkdir("/q");
- d) create("/q");
- e) None of the above

16) File System State after **TWO** operations:

```
inodes [d a:0 r:4] [d a:1 r:4] [f a:-1 r:2] [f a:-1 r:1] [] [] data
[(.,0) (.,0) (c,1) (r,2)] [(.,1) (.,0) (q,2) (j,3)] [] []
```

- a) creat("/c/j"); link("/c/q", "/r");
- b) creat("/c/j"); creat("/r");
- c) creat("/r"); link("/c/q", "/c/j");
- d) mkdir("/j"); link("/c/q", "/r");
- e) None of the above

17) File System State after **TWO** operations:

```
inodes [d a:0 r:5] [d a:1 r:4] [f a:2 r:2] [f a:-1 r:1] [f a:-1 r:1]
data [(.,0) (.,0) (c,1) (r,2) (g,4)] [(.,1) (.,0) (q,2) (j,3)] [v]
```

- a) creat("/g");
fd=open("/c/q", O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE); close(fd); b)
creat("/c/g");
fd=open("/c/q", O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE); close(fd);
- c) creat("/c/g");
fd=open("/c/g", O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE); close(fd);
- d) creat("/g"); creat("/c/q");
- e) None of the above.

Links [2 points each]

Imagine the following commands are run on an FFS-like file system that supports hard and soft links.

```
echo "file" > file
ln file file2
echo "file2" >> file2
mv file file3
echo "file3" >> file3
```

- 18) What output will you see if you run `cat file2` (assume nothing wrong with whitespace or carriage returns)?
- a) file2\n
 - b) file\n file2\n
 - c) file\n file2\n file3\n
 - d) file2: No such file or directory
 - e) None of the above

- 19) What output will you see if you run `cat file3` (assume nothing wrong with whitespace or carriage returns)?
- a) file3\n
 - b) file\n file3\n
 - c) file\n file2\n file3\n
 - d) file3: No such file or directory
 - e) None of the above

Imagine the following commands are run instead on an FFS-like file system:

```
echo "bar" > bar
ln -s bar bar2
echo "bar2" >> bar2
```

```
mv bar bar3
echo "bar3" >> bar3
```

20) What output will you see (assume nothing wrong with whitespace or carriage returns) if you run `cat bar2` ?

- a) bar2\n
- b) bar\n bar2\n
- c) bar\n bar2\n bar3\n
- d) bar2: No such file or directory
- e) None of the above

21) What output will you see (assume nothing wrong with whitespace or carriage returns) if you run `cat bar3` ?

- a) bar3\n
- b) bar\n bar3\n
- c) bar\n bar2\n bar3\n
- d) bar3: No such file or directory
- e) None of the above

Crash Consistency [2 points each]

Imagine you have an FFS-like file system that is creating a new empty file in an existing directory and must update 4 blocks: the directory inode, the directory data block, the file inode, and the inode bitmap. Assume the directory inode and the file inode are in different on-disk blocks. Assume this initial system does not perform any journaling and FSCK is not run. What happens if a crash occurs after only updating the following block(s)?

22) Bitmap

- a) No inconsistency (it simply appears that the operation was not performed)
- b) Data/inode leak
- c) Multiple file paths may point to same inode
- d) Point to garbage
- e) Multiple problems listed above

23) File inode

- a) No inconsistency (it simply appears that the operation was not performed)
- b) Data/inode leak
- c) Multiple file paths may point to same inode
- d) Point to garbage
- e) Multiple problems listed above

24) Directory inode and Directory data

- a) No inconsistency (it simply appears that the operation was not performed)
- b) Data/inode leak
- c) Multiple file paths may point to same inode
- d) Point to garbage
- e) Multiple problems listed above

25) Bitmap and File inode

- a) No inconsistency (it simply appears that the operation was not performed)
- b) Data/inode leak
- c) Multiple file paths may point to same inode
- d) Point to garbage
- e) Multiple problems listed above

26) Bitmap and Directory inode and Directory Data

- a) No inconsistency (it simply appears that the operation was not performed)
- b) Data/inode leak
- c) Multiple file paths may point to same inode
- d) Point to garbage
- e) Multiple problems listed above

27) File inode and Directory inode and Directory Data

- a) No inconsistency (it simply appears that the operation was not performed)
- b) Data/inode leak
- c) Multiple file paths may point to same inode
- d) Point to garbage
- e) Multiple problems listed above

Assume we've added a basic implementation of full-data journaling to our FFS-like file system and perform the same **file create** operation as above. Assume a **transaction header** block and a **transaction commit** block. Assume each block is written synchronously (i.e., a barrier is performed after every write and blocks are pushed out of the disk cache). If the system crashes after the following number of blocks have been synchronously written to disk, what will happen after the system reboots? (If the number of disk writes exceeds those needed, assume they are unrelated.)

1) 1 disk write (hint: just the transaction header block is written to disk)

- a) No transactions replayed during recovery; file system in old state
- b) No transactions replayed during recovery; file system in new state
- c) Transaction replayed during recovery; file system in old state
- d) Transaction replayed during recovery; file system in new state
- e) Transaction replayed during recovery; file system in unknown state

2) 4 disk writes (hint: transaction header, plus 3 blocks to journal)

- a) No transactions replayed during recovery; file system in old state
- b) No transactions replayed during recovery; file system in new state
- c) Transaction replayed during recovery; file system in old state
- d) Transaction replayed during recovery; file system in new state
- e) Transaction replayed during recovery; file system in unknown state

3) 5 disk writes (hint: transaction header, plus 3 blocks to journal, plus ???)

- a) No transactions replayed during recovery; file system in old state

- b) No transactions replayed during recovery; file system in new state
 - c) Transaction replayed during recovery; file system in old state
 - d) Transaction replayed during recovery; file system in new state
 - e) Transaction replayed during recovery; file system in unknown state
- 4) 6 disk writes
- a) No transactions replayed during recovery; file system in old state
 - b) No transactions replayed during recovery; file system in new state
 - c) Transaction replayed during recovery; file system in old state
 - d) Transaction replayed during recovery; file system in new state
 - e) Transaction replayed during recovery; file system in unknown state
- 5) 8 disk writes
- a) No transactions replayed during recovery; file system in old state
 - b) No transactions replayed during recovery; file system in new state
 - c) Transaction replayed during recovery; file system in old state
 - d) Transaction replayed during recovery; file system in new state
 - e) Transaction replayed during recovery; file system in unknown state
- 6) 10 disk writes
- a) No transactions replayed during recovery; file system in old state
 - b) No transactions replayed during recovery; file system in new state
 - c) Transaction replayed during recovery; file system in old state
 - d) Transaction replayed during recovery; file system in new state
 - e) Transaction replayed during recovery; file system in unknown state
- 7) 11 disk writes
- a) No transactions replayed during recovery; file system in old state
 - b) No transactions replayed during recovery; file system in new state
 - c) Transaction replayed during recovery; file system in old state
 - d) Transaction replayed during recovery; file system in new state
 - e) Transaction replayed during recovery; file system in unknown state

Tru

True/False about Persistence [3 points each]

Designate if the statement is True (a) or False (b).

- 33) When interacting with a device, it is usually better to use **PIO** than **DMA**.
- 34) **Device driver code** is software that executes on the microcontroller of a peripheral device. *False*;
- 35) With a 15000 RPM disk, the **expected rotation time** for a random access is 4 ms.
- 36) **Transfer time** for disk sectors is significantly longer for random accesses than for sequential accesses.
- 43) RAID-0 is also referred to as **striping**.

- 44) With **RAID-1**, the steady-state throughput of random reads and random writes are identical. *False*;
- 45) RAID-4 has better **capacity** than RAID-1 and better **reliability** than RAID-0.
- 46) One disadvantage of RAID-4 is that it cannot continue operating if the **single parity disk** fails.
- 47) The **capacity** of RAID-4 and RAID-5 systems are identical.
- 48) The steady-state throughput for **random writes** with RAID-4 is half the steady-state throughput of random writes on a single disk.
- 49) For **random read** operations, RAID-1 delivers better throughput than RAID-5.
- 51) In an FFS-like file system, multiple inodes may point to the **same file descriptor**.
- 52) In an FFS-like file system, multiple inodes may point to the **same path name**.
- 53) An advantage of soft links over hard links is that soft links can be used to refer to **directories**.
- 54) **Contiguous allocation** of files to blocks on disk tends to achieve excellent sequential bandwidth.
- 55) A **File-Allocation Table** suffers from external fragmentation.
- 56) An inode structure typically contains a field indicating the **name** of this file.
- 57) An inode structure typically contains a field indicating the **time** at which this file was last accessed.
- 58) An inode structure can contain **pointers to directory data**.
- 59) FFS attempts to put the **inode and data blocks** from the same file in the same cylinder group.
- 60) FFS attempts to put **inodes from files in the same directory** in the same cylinder group.
- 61) When placing parts of a **large file** in a “new” group, FFS looks for a group with more than the average number of free inodes.
- 62) After a crash, **FCK** fixes the on-disk state of the file system to match its state before the last update to disk began.
- 63) After the journal **commit** block is successfully written to disk, a journaling file system can then **checkpoint** the relevant blocks to their final in-place positions.
- 64) **Write-back journaling** forces data blocks to be written to their final in-place positions before

committing a journal transaction that refers to those data blocks.

- 65) When LFS writes a new copy of a **data block** to a segment, it also writes a new copy of the **inode** that points to that data block.
- 66) When LFS writes a new copy of **inode** to a segment, it also writes a new copy of the **directory** that points to that inode.
- 67) LFS periodically **checkpoints imaps** to a known location on disk (alternating between two locations to withstand crashes).
- 68) When performing **garbage collection**, LFS determines that an **inode** is valid by verifying that the corresponding entry in the imap points to this location.
- 69) When performing **garbage collection**, LFS determines that a **data block** is valid by scanning all valid inodes from the imap and verifying that one of the valid inodes points to this location.